Home Hazard Report — PRD (DubHacks 2026)

Oct 16, 2026 · @Turker

How to use this doc with Cursor

Export this doc as Markdown, save it as `docs/PRD.md` in the repo, and tell Cursor: "Read docs/PRD.md before every task. Build in the order of the Build order section." Rules for the AI agent:

1. Never invent hazard data, dates, numbers, or sources. Every number shown to a user must come from a data file or API response.
2. Build one P0 feature end to end before starting the next. A working thin slice beats three half-built ones.
3. Every hazard result must carry its source, observation date(s), resolution, and limitations (see the response schema).
4. "No data" and "no hazard" are different results. Never collapse them into one.
5. Keep secrets server-side. The xAI API key never reaches the browser; the browser gets an ephemeral token.
6. Small pull requests, one feature each, so VEYZL audits run per feature.
7. When something in this doc is marked VERIFY, check it against the real data or docs before relying on it.

Overview

A resident types a King County address and gets a report on three hazards at that exact spot (flooding, heat, ground motion), each backed by observed NASA satellite data, not models. Grok turns the numbers into plain language, and the user can ask follow-up questions out loud with Grok Voice.

Problem. People choose where to live with almost no hazard information. Renters get the least. In December 2025 the lower Snoqualmie River had its flood of record, and many residents had no clear way to know their spot's flood exposure beforehand.

Goals for the hackathon

1. Any King County address returns a complete report in under 10 seconds.
2. Every hazard card shows what satellites observed, when, at what resolution, and what the data cannot tell you.
3. Every card ends with one concrete action (sign up for alerts, find a cooling center, check the foundation).
4. Voice answers come from the same data as the cards, never from the model's general knowledge.

Success at judging. A judge enters their own address and gets a working report. The demo shows the December 10, 2025 flood near Carnation, a heat contrast between two neighborhoods, and one spoken follow-up question.

Users and core flow

Primary user: a King County resident deciding where to live, especially a renter.
Secondary: current residents checking their own home, and community groups doing outreach.

Core flow

1. User types an address in the search bar and submits.
2. App geocodes it. If it's outside King County, show a friendly message and stop.
3. App shows a map centered on the address and three hazard cards loading in parallel: Flood, Heat, Ground motion.
4. Each card fills in independently. One slow or failed layer never blocks the others.
5. A short Grok-written summary appears above the cards once the hazard results are in.
6. Each card has a "How we know this" panel the user can expand.
7. User taps the microphone and asks a question, such as "Should I worry about my basement?" Grok Voice answers out loud using this address's results.

Scope and priorities

P0 is the demo; nothing in P1 starts until every P0 item works end to end.

Priority	Feature	Notes
P0	Address search + geocoding	US Census geocoder, King County check

Priority	Feature	Notes
P0	Flood card (flood history)	DSWx-S1 history map; live NASA service as fallback
P0	Heat card	Landsat surface temperature, compared to the county
P0	Map + 3 cards + "How we know this"	Loading, error, and no-data states for each card
P0	Grok written summary	Generated only from the hazard JSON
P0	Grok Voice follow-up	Function call to the hazards endpoint; required for SpaceXAI
P0	Deployed public URL	Judges try their own addresses
P1	Ground motion card	Built and validated; Seattle, Eastside and Snoqualmie Valley only
P2	FEMA flood zone beside observed flooding	Only after all three cards work
P2	Shareable report link	Supports the SideShift pitch

Out of scope: user accounts, saved searches, areas outside King County, forecasts or predictions, earthquake shaking, mobile app, payments.

Hackathon constraints and tracks

Hacking runs from 12:30 PM Saturday to a hard 10:00 AM Sunday deadline, about 21.5 hours. A Devpost draft is due Saturday at 10:30 PM. Teams without a video are not judged.

Track	What it requires from the build
Community Kitchen (main)	Framing for residents and renters; equity; an action on every card. Judged on functionality 30%, track fit 30%, design 20%, creativity 20%.
SpaceXAI	Built in Cursor (use it heavily); uses Grok Voice API; real space data. Judged only from Devpost and the video.

Track	What it requires from the build
VEYZL	GitHub repo connected; audit runs on pull requests; show a finding that changed the code, with before and after.
SideShift	Business case and distribution story; same product, pitched as a report people share.

Build implications: use Cursor for all coding; keep Grok Voice working through the end; open small pull requests from hour one so VEYZL has history; record the video by 7:00 AM Sunday.

Architecture and tech stack

Two services in one repo: a Next.js web app and a Python FastAPI backend. All satellite processing happens offline in scripts; at request time the backend only reads one pixel from prepared map files, so the app never depends on a NASA server during judging.

Part	Technology	Responsibility
Web app	Next.js (App Router, TypeScript), Tailwind, shadcn/ui, MapLibre GL	UI, map, cards, Grok report call, ephemeral token route, voice client
Backend	Python 3.11+, FastAPI, rasterio, pyproj, numpy	Geocoding, hazard lookups, JSON responses
Data scripts	Python, earthaccess, rasterio, xarray, pystac-client, planetary-computer	One-time downloads and processing into GeoTIFFs
AI	xAI Grok API (text) and Grok Voice (realtime WebSocket)	Written summary; spoken follow-ups
Hosting	Vercel (web), Render or Railway (backend)	Public URLs for judges

Request flow

1. Browser sends the address to `GET /api/report?address=...` on the web app.
2. The web app calls the backend `GET /hazards?address=...`.
3. Backend geocodes, checks the point is in King County, then reads the flood, heat, and ground-motion map files at that point.

4. Backend returns one JSON object with all three hazards (schema below).
5. The web app renders the cards, then sends that JSON to Grok for the written summary.
6. For voice, the browser gets an ephemeral token from `POST /api/voice-token`, opens the Grok Voice WebSocket, and Grok calls the `get_hazards` function, which the browser answers with the same JSON.

Data sources and pipelines

Three NASA-backed layers, each processed once into a GeoTIFF the backend reads at request time. Catalog checks below were run on October 16, 2026.

Layer	Product	Coverage	Output file
Flood history	NASA JPL OPERA DSWx-S1 (`OPERA_L3_DSWX-S1_V1`), 30 m	1,158 radar passes over King County, Sep 2024 to Oct 15, 2026; tiles T10UFU, T10TFT, T10TET	`data/flood/flood_history_<tile>.tif`
Heat	Landsat 8/9 Collection 2 Level-2 surface temperature (NASA/USGS), 30 m delivered, 100 m native	About 45 clear summer days (June to September), 2013 to 2025; path/row 046/027 covers the whole county	`data/heat/heat_history.tif` , `data/heat/heat_stats.json` , `data/king_county.geojson`

Layer	Product	Coverage	Output file
Ground motion	NASA JPL OPERA DISP-S1 (`OPERA_L3_DISP-S1_V1`), 30 m	983 granules over King County; frames F03321, F03322, F36550, F16950, F16951, F30707, F30708	`data/ground_motion/ground_motion.tif`

Geocoding

Use the US Census geocoder (free, no key). `GET https://geocoding.geo.census.gov/geocoder/locations/onelineaddress?address=<text>&benchmark=Public_AR_Current&format=json` returns `result.addressMatches[0].coordinates` with `x` = longitude and `y` = latitude (tested: 400 Broad St, Seattle returned -122.3489, 47.6198). For the King County check, VERIFY whether the `/geocoder/geographies/onelineaddress` variant returns the county in one call; otherwise use a point-in-polygon test against a King County boundary GeoJSON stored in `data/`.

Flood history (P0)

Status: built and validated October 16, 2026. `scripts/build_flood_history.py` makes the dataset; `backend/flood_lookup.py` reads one point (set `HISTORY_DIR` to the repo's `data/` folder). Validation: Carnation (47.664, -121.920) = `flooded`, 281 valid passes, wet once on 2025-12-10; Seattle Center = `no_flooding_seen`, 196 passes; Lake Washington = `permanent_water`, wet on 281 of 281.

1. Search `OPERA_L3_DSWX-S1_V1` with earthaccess over King County, September 1, 2024 to today, all months. Keep only `_B01_WTR.tif` files; when a tile and acquisition time appear twice, keep the newest production. Result: 1,158 files, 621 MB. Raw files stay on the data owner's laptop, never in the repo.
2. Pixel values: 0 = not water, 1 = open water, 3 = flooded vegetation; anything else (250, 251, fill) = no valid reading.

3. For each tile and pixel, count valid passes and wet passes, and record first and last wet date. Acquisition times are converted from UTC to Pacific before taking the date.
4. Classify at lookup, as a share of valid passes: fewer than 10 valid = `not_enough_data`; wet on 50% or more = `permanent_water`; 5% or more = `frequently_wet`; wet at least once = `flooded`; otherwise `no_flooding_seen`. The 5% and 50% thresholds are starting guesses; tune them against a farm field, a parking lot or runway, and a riverside neighborhood.
5. Open decision: the lookup reads one 30 m pixel. Consider also reporting "water seen within 100 m of this address" to absorb geocoder error.

Fallback if the history build fails: the live NASA Disasters map service for the December 2025 event, `https://gis.earthdata.nasa.gov/portal/rest/services/DISASTERS_202512_FLOOD_WA/2512_opera_dswx_s1/MapServer/identify`. Layer IDs: 16 = Nov 28, 13 = Dec 6, 11 = Dec 10, 10 = Dec 13. Response field `attributes["UniqueValue.Pixel Value"]`; `NoData` means that pass did not cover the point. This service already moved once, so cache responses for demo addresses.

Heat (P0)

1. `scripts/build_heat.py` makes the dataset; `backend/heat_lookup.py` reads one point (set `DATA_DIR` to the repo's `data/` folder). No login needed. Source: Microsoft Planetary Computer STAC, collection `landsat-c2-l2`, Landsat 8 and 9, path 046 row 027, June to September, scene cloud cover under 10%, 2013 to 2025.
2. Per day: read `lwir11`, convert to °F with the asset's scale 0.00341802 and offset 149 (verified), and mask with `qa_pixel` (fill, dilated cloud, cirrus, cloud, shadow, snow, water). Skip a day if under 50% of lowland King County is clear.
3. Relative heat: on each day, subtract the median of lowland King County (county land west of longitude -121.9). This removes daily weather, so days and years can be compared. Daily lowland medians ranged from 72°F to 100°F in 2024 to 2025 alone.
4. `heat_history.tif` (int16, tenths of °F): band 1 recent temperature (2023 to 2025), band 2 recent relative heat, band 3 early relative heat (2013 to 2016), band 4 clear days used, bands 5 to 17 relative heat per year 2013 to 2025. `heat_stats.json` holds lowland percentiles of recent relative heat and every date used. Per-day files (`heat_cache/`) stay on the laptop.

5. Lookup averages a 5×5 block (150 m) and returns recent temperature, relative heat, lowland percentile and label, change since 2013 to 2016 (labeled a change only at 3°F or more), and the yearly series.
6. Tested October 17 on 2024 to 2025 data: SODO 105.8°F, +18.6°F, 98th percentile; Discovery Park 78.1°F, -9.0°F, 21st percentile; Lake Washington `not_enough_data` (water excluded).

The satellite passes at about 18:55 UTC, about 11:55 AM Pacific in summer. Values are ground and roof surface temperature at that time, not air temperature. In the UI, draw the yearly series as a small line chart on the heat card.

Ground motion (built)

Status: built and validated October 17, 2026. `scripts/build_ground_motion.py` makes the dataset; `backend/ground_lookup.py` reads one point (`DATA_DIR` points at `data/ground_motion/`). Validation: UW east campus at 47.65626, -122.2981 (reportedly the former Montlake Landfill; verify before saying so) = -15.8 mm/yr, about -139 mm from 2016 to June 2025, matching the ASF Displacement Portal; Downtown Bellevue = 0.1 mm/yr, stable.

1. Source: `OPERA_L3_DISP-S1_V1` , frame F16951, variable `short_wavelength_displacement` (the layer ASF's portal uses), read remotely over HTTPS. The Earthdata account must have authorized ASF. Window: longitude -122.45 to -121.90, latitude 47.38 to 47.78 (Seattle, the Eastside, the Snoqualmie Valley, south to about Renton).
2. Chaining: each file holds displacement since its ministack's reference date, which resets about yearly. The build uses one file per ministack, the one ending on the next reference date: 8 of 119 files, giving 9 dates from 2016-07-07 to 2025-10-12.
3. Velocity = total measured movement / total measured time over the segments that have data. A pixel needs at least 6 of 8 segments and must pass NASA's `recommended_mask` . 35% of the window qualifies; water, forest and downtown Seattle's high-rises have no reliable data.
4. `ground_motion.tif` (float32): band 1 velocity in mm/yr, band 2 valid flag, bands 3 to 11 cumulative mm at each date. `ground_stats.json` lists the dates.
5. Lookup averages a 3×3 block (90 m) and needs at least 3 reliable pixels. Under 3 mm/yr either way = stable. Negative = moving away from the satellite, consistent with sinking. Returns velocity, estimated total change, the dated series and the period.

6. Outside the window (Kent, Auburn, Federal Way, Enumclaw, North Bend, most of Vashon) the lookup returns `outside_coverage`; the card says "not covered yet". Other DISP-S1 frames (F03321, F30707, F36550) could extend coverage later at about 10 minutes each.

Backend API

One main endpoint returns everything the UI and voice need; each hazard is computed independently so one failure never breaks the response.

Method	Path	Purpose
GET	`/hazards?address=<text>`	Geocode, then return all three hazards
GET	`/hazards?lat=<f>&lon=<f>`	Same, from coordinates (map clicks, testing)
GET	`/health`	Returns which data files are loaded

Response shape (every hazard uses the same `status` + `source` structure):

```
{
  "address": "123 Example Rd, Carnation, WA 98014",
  "lat": 47.664, "lon": -121.92,
  "in_king_county": true,
  "hazards": {
    "flood": {
      "status": "flooded",
      "observed_passes": 281, "wet_passes": 1,
      "first_wet": "2025-12-10", "last_wet": "2025-12-10",
      "source": {
        "name": "NASA JPL OPERA DSWx-S1",
        "dates": "All radar passes, Sep 2024 to Oct 2026",
        "resolution_m": 30,
        "limitations": ["Passes are days apart and can miss a flood's peak",
"Radar misses water under trees and tall crops", "Very smooth surfaces can
look like water"]
      }
    },
    "heat": {
      "status": "ok",
      "recent_surface_temp_f": 0, "recent_relative_f": 0,
"lowland_percentile": 0,
```

```
      "label": "hotter than most of lowland King County",
      "change_since_early_f": 0, "change_label": "about the same as it used
to be",
      "yearly_relative_f": [{ "year": 2013, "relative_f": 0 }],
      "clear_days_used": 0,
      "source": { "name": "Landsat 8/9 Collection 2 surface temperature
(NASA/USGS)", "dates": "Clear summer days, 2013 to 2025", "resolution_m": 30,
"limitations": ["Ground and roof temperature around noon, not air or indoor
temperature", "Clear days only"] }
    },
    "ground_motion": {
      "status": "not_enough_data",
      "velocity_mm_per_yr": null,
      "source": { "name": "NASA JPL OPERA DISP-S1", "dates": "",
"resolution_m": 30, "limitations": ["Measured along the satellite's slanted
view, not straight up and down", "Slow movement only, not earthquake
shaking"] }
    }
  }
}
```

The zeros above are placeholders, not real values.

Status values: flood = `flooded` | `frequently_wet` | `no_flooding_seen` | `permanent_water` | `not_enough_data` | `error`. Heat and ground motion = `ok` | `not_enough_data` | `error` | `not_available` (ground motion when the layer is cut).

Errors: address not found → 404 with a friendly message; outside King County → 200 with `in_king_county: false` and no hazards; a failing layer → that hazard's `status: "error"`, others still returned.

AI: Grok summary and Grok Voice

Both AI features read only the hazards JSON; neither may state a hazard fact that isn't in it.

Written summary

Called server-side from the web app after `/hazards` returns. Use the xAI API with the current recommended text model (VERIFY the model name and endpoint at docs.x.ai). System prompt:

```
You write a short hazard summary for a resident of the address in the JSON.
Rules:
- Use only facts in the JSON. Never add hazard facts, numbers, or dates.
- 3 to 5 sentences, plain language, 8th-grade reading level.
- Lead with the most significant hazard.
- If a status is not_enough_data or error, say the data could not tell us,
not that the spot is safe.
- "no_flooding_seen" means no flooding was observed on satellite passes, not
that the home cannot flood.
- End with the single most useful action from the actions provided.
```

Voice follow-ups

- Endpoint: `wss://api.x.ai/v1/realtime?model=grok-voice-latest`.
- Browsers cannot send WebSocket headers, so `POST /api/voice-token` (server-side) creates an ephemeral token, and the browser connects with the subprotocol `xai-client-secret.<token>`. Never ship the API key to the client.
- Start from xAI's cookbook example `examples/voice-agent-web` (github.com/xai-org/xai-cookbook).
- On open, send `session.update` with: a voice, `turn_detection: {"type": "server_vad"}`, PCM 24 kHz audio, the instructions below, and one function tool `get_hazards` (no parameters; returns the current address's hazards JSON).
- Function flow: on `response.function_call_arguments.done`, send `conversation.item.create` with a `function_call_output` holding the JSON, wait for current audio to finish, then send `response.create`.
- Test `reasoning.effort` at `none` versus the default `high` and keep whichever feels fast enough on stage.

Voice instructions:

```
You answer spoken questions about natural hazards at one home address.
Always call get_hazards before answering a question about this home.
Use only that data. If the data cannot answer the question, say so and
suggest who could (King County flood warning center, a structural engineer,
the landlord).
Keep answers under 4 sentences. Never claim a home is safe. Do not give
legal, insurance, or medical advice.
```

Frontend

One page, readable in five seconds: address bar on top, map on the left, summary and three cards on the right (stacked on mobile).

Components

- `AddressSearch`: input + submit; Enter key works; disabled while loading.
- `HazardMap` (MapLibre): pin at the address; toggle for the flood layer and the heat layer as raster overlays if time allows.
- `Summary`: the Grok paragraph, with a skeleton while it loads.
- `HazardCard` (one component, used three times): title, a one-line finding, a severity label, the action, and a collapsible "How we know this" panel showing source name, dates, resolution, and limitations.
- `VoiceButton`: mic toggle with listening, thinking, and speaking states, plus a live transcript.

Card states (every card must handle all of these):

State	What the card shows
Loading	Skeleton, never a blank space
Hazard found	Finding, severity, action
No hazard seen	"Not observed in N satellite passes", never "safe"
Not enough data	"Satellites couldn't get a reliable reading here"
Error	"This layer is unavailable right now"; the other cards still show

Actions per card: flood → King County flood alert sign-up (VERIFY the current link); heat → cooling centers and ways to keep a home cool; ground motion → watch for new foundation or wall cracks and ask the landlord or an inspector.

Design: use shadcn/ui components, one accent color per hazard, plain language, no jargon in the main card text (technical terms go only in "How we know this").

Repo structure and environment

One GitHub repo, connected to VEYZL from the first commit.

```
/
├─ docs/PRD.md                  # this document
├─ client/                      # Next.js app (Vercel), already initialized
|  ├─ app/page.tsx
|  ├─ app/api/report/route.ts   # calls backend, then Grok summary
|  ├─ app/api/voice-token/route.ts
|  └─ components/ (AddressSearch, HazardMap, Summary, HazardCard,
VoiceButton)
├─ backend/                     # FastAPI (Render or Railway)
|  ├─ main.py                  # /hazards, /health (next to build)
|  ├─ flood_lookup.py          # done: flood_history(lat, lon)
|  ├─ heat_lookup.py           # done: heat_at(lat, lon)
|  ├─ ground_lookup.py         # done: ground_at(lat, lon)
|  └─ requirements.txt
├─ scripts/                     # one-time data processing, run locally
(done)
|  ├─ build_flood_history.py
|  ├─ build_heat.py
|  └─ build_ground_motion.py
└─ data/                        # summary files only, never raw downloads
   ├─ flood/          flood_history_T10TET.tif, flood_history_T10TFT.tif,
flood_history_T10UFU.tif
   ├─ heat/           heat_history.tif, heat_stats.json
   ├─ ground_motion/  ground_motion.tif, ground_stats.json
   └─ king_county.geojson       # county boundary for the in-county check
```

Variable	Where	Purpose
`XAI_API_KEY`	web (server only)	Grok summary and voice tokens
`BACKEND_URL`	web	Address of the FastAPI service
`EARTHDATA_USERNAME`, `EARTHDATA_PASSWORD`	local scripts only	NASA downloads
`DATA_DIR`	backend	Folder holding the GeoTIFFs

Add `.env*` and the raw download folders to `.gitignore` before the first commit. The generated GeoTIFFs ship with the backend; if they are too large for git, store them in a release asset or object storage and download at startup.

Build order and timeline

The first goal is a thin end-to-end slice by 6 PM Saturday: address in, flood card out, deployed. Everything else layers on top.

Status, Saturday 2 PM: all three data layers (flood, heat, ground motion) are built, validated and committed on the `data-layers` branch. The lookups already point at `data/flood/`, `data/heat/` and `data/ground_motion/`; do not change those paths. Next on the critical path: `backend/main.py` with `/hazards`, then deploy, then the cards.

Time	Data (Turker)	AI and voice	Frontend	Integration
Sat 12:30–1:30 PM	Start flood download	Voice cookbook running locally	Page skeleton, shadcn, MapLibre	Repo, VEYZL, deploy hello-world to Vercel + Render
1:30–6:00 PM	Flood history built and validated; start heat	`/api/voice-token`, voice connects	Address bar, map pin, `HazardCard` with all states	`/hazards` with geocoding + flood lookup
6:00 PM	**Gate:** flood card works end to end on the deployed URL; decide ground motion go/no-go			
6:00–10:30 PM	Heat layer done; ground motion if go	Grok summary; `get_hazards` function call	Summary, "How we know this", polish	Heat lookup wired in; Devpost draft submitted by 10:00
10:30 PM	**Gate:** ground motion working or cut			
10:30 PM–3:00 AM	Ground motion (if still go)	Voice polish; fix VEYZL findings via PRs	Mobile layout, loading states	Error handling, caching demo addresses

Time	Data (Turker)	AI and voice	Frontend	Integration
3:00–7:00 AM	Bug fixes, sleep in shifts. Feature freeze at 7:00 AM			
7:00–9:30 AM	Record 3-minute video; finish Devpost (data sources, Cursor use, AI disclosure); submit by 9:30			
10:20 AM	Whole team at the HUB Ballroom			

If the flood history build is not validated by 5 PM, switch the flood card to the live December 2025 fallback and move on.

Acceptance criteria and demo

The build is demo-ready when every box below is ticked on the deployed URL, not localhost.

- [ ] 47.664, -121.920 (near Carnation) shows flood status `flooded` with 2025-12-10 as a wet date
- [ ] A Lake Washington point shows `permanent_water`, not `flooded`
- [ ] A downtown Seattle address returns all cards without errors
- [ ] An address outside King County shows a friendly message, not a crash
- [ ] A misspelled or fake address shows "address not found"
- [ ] Killing one data file makes only that card show its error state
- [ ] Full report loads in under 10 seconds on venue WiFi
- [ ] Grok summary contains no number or date that isn't in the hazards JSON (check 5 addresses)
- [ ] Voice answers "Did this place flood?" using `get_hazards`, out loud, within about 3 seconds
- [ ] API key is not visible in browser dev tools

- [ ] At least one VEYZL finding fixed in a PR, with before and after screenshots
- [ ] Backup screen recording of the full demo saved locally

3-minute demo

1. Problem, 30 seconds: the December 2025 flood of record on the lower Snoqualmie, and why residents, especially renters, lacked information.
2. Live demo, 90 seconds: Carnation address → flood card with the December 10 observation; a second address for heat contrast; one spoken question to Grok Voice.
3. How we know, 30 seconds: observed satellite records, not predictions; open one "How we know this" panel.
4. Impact and next steps, 30 seconds: renters, alerts, more counties, ground motion if it was cut.

Keep separate 60-second versions for VEYZL (the finding, the fix, why it matters) and SideShift (who pays, how it spreads).

Limitations and wording rules

Owning the limits is part of the creativity score; these rules apply to every card, the summary, voice, and the pitch.

Never say	Say instead
"This home is safe"	"No flooding was observed in N satellite passes"
"Flood history since 2009"	"Radar passes since September 2024"
"Record flood in Carnation"	"Flood of record on the lower Snoqualmie" (King County's wording)
"Air temperature" or "indoor heat"	"Ground and roof surface temperature around noon on clear summer days"
"The ground is sinking X mm/yr" without context	"Moved about X mm/yr toward or away from the satellite" (after the sign is verified)
"Earthquake risk"	"Slow ground movement"
"Not flooded" for a pixel with no reading	"Satellites couldn't get a reliable reading here"

Known data limits to show in "How we know this":

- Flood: radar passes are days apart and the December 10 pass came about 10 hours before the river peaked; radar misses water under trees and tall crops; very smooth surfaces (runways, large lots, flat roofs) can look like water, so a single wet pass in a city may be a false alarm.
- Heat: one time of day, clear days only, 100 m native thermal detail.
- Ground motion: slanted measurement direction; unreliable in forests and farmland; reference dates change over time.

Only claim an equity pattern (for example, hotter lower-income neighborhoods) if your own processed data shows it.